Практикум на языке программирования Лисп
(3-й курс, осень 2025 г.)

Задание 1. Программирование вспомогательных функций

- Реализовать с помощью базового Лиспа (набор из восьми встроенных функций), а также встроенных арифметических функций + и - и функций member и eql:
  I. 2 функции - по1 из раздела Ia и Ib;
  II. 3 любые функции из раздела II;

I. Простая рекурсия

`L` – обозначение произвольного списка, `N` – целое число.

a)

`(pair L)` разбивает элементы списка L на пары:

```
(pair '(a b c d e)) => ((a b)(c d)(e nil))
```

`(mix2 L1 L2)` образует список пар элементов, взятых последовательно из заданных списков:

```
(mix2 '(a b c) '(z x)) => ((a z)(b x)(c nil))
```

b)

`(elem N L)` выбирает на верхнем уровне списка `L` |`N`|-тый элемент, отсчитанный от начала списка, если `N>0`, или от конца списка, если `N<0`, например:

```
(elem 2 '(a (12) b (3))) => (12)
(elem -2 '(a (12) b (3)) => b
```

`(rest N L)` выдает список, полученный из списка `L` отбрасыванием `N` первых элементов списка верхнего уровня, если `N>=0`, или |`N`| последних элементов, если `N<0`, например:

```
(rest 1 '(a (12) b (3))) => ((12) b (3))
(rest -2 '(a (12) b (3))) => (a (12))
```

`(head N L)` вырабатывает список, составленный из `N` первых элементов верхнего уровня списка `L`, если `N>=0`, или |`N`| последних элементов, если `N<0`, например:

```
(head 1 '(a (12) b (3))) => (a)
(head -2 '(a (12) b (3))) => (b (3))
(head 0 '(a (12) b (3))) => ()
```

Замечание: в функциях `elem`, `rest`, `head` значение первого аргумента `N` должно быть не больше длины второго аргумента – списка `L`.

II. Параллельная рекурсия

`S` обозначает произвольное списочное выражение, а `N` – целое число.

`(level N S)` строит список из элементов списочного выражения `S`, находящихся на уровне `N` (`N>=1`), например:

```
(level 2 '(((a(5)8) b)7 (g(())) )) => ((a(5)8) b g(nil))
```

`(trans N S)` упрощает структуру списочного выражения `S`, заменяя в нем все списочные элементы, находящиеся на уровне `N` (`N>=1`) на атом “#”, например:

```
(trans 2 '(((a(5)8) b (k))(g (c)))) => ((# b #)(g #))
(trans 3 '( ( (a (5) 8) b ((f) v) (k)) (g (c) ((g)))))
```

`(deep S)` определяет атом в списочном выражении `S`, который находится на наиболее глубоком уровне (если таких атомов несколько, выбрать любой), например:

```
(deep '(a (b (a)) c ((d) 8))) => a
```

`(freq S)` для каждого входящего в списочное выражение `S` атома вычисляет частоту его вхождения и строит список, состоящий из списков пар: атом – его частота, например:

```
(freq '(a (b (a)) c ((a) 8))) => ((a 3)(b 1)(c 1)(8 1))
```

`(flat S)` вырабатывает `t` тогда, когда список `S` является выровненным, например:

```
(flat '(((b) (a c)) ((c 5)) ((d) 8))) => nil
(flat '(((b) (a c)) ((c 5)) ((d)))) => t
```

Выровненным считается одноуровневый список из атомов, а также любой список, состоящий из элементов с одинаковой глубиной (глубина списка есть максимальное количество уровней в нём).